



Instituto Tecnológico de Matamoros

Verano De Investigacion Delfin 2026

Evaluación experimental de vulnerabilidades en APIs bajo un enfoque DevSecOps

Reporte técnico de prácticas de laboratorio

Autor: Miguel Angel Davila Hernandez

Asesor: Dr. José Antonio Vergara Camacho

Institución: Instituto Tecnológico de Matamoros

Periodo: Junio-julio de 2026

Julio de 2026

Índice

1. Resumen	1
2. Introducción	1
3. Objetivos	1
3.1. Objetivo general	1
3.2. Objetivos específicos	1
4. Metodología general	2
5. Laboratorio 1: Evaluación de seguridad de un API Gateway	2
5.1. Objetivo y alcance	2
5.2. Herramientas y entorno de ejecución	3
5.3. Procedimiento experimental	3
5.3.1. Despliegue de los servicios	3
5.3.2. Publicación temporal del API Gateway	3
5.3.3. Ejecución de los escaneos	5
5.4. Resultados y evidencias	5
5.4.1. Resultado de MDN HTTP Observatory	5
5.4.2. Resultado de SecurityHeaders.com	5
5.5. Hallazgos e impacto técnico	6
5.6. Mitigación aplicada	6
5.6.1. Configuración inicial vulnerable	6
5.6.2. Configuración corregida	8
5.7. Conclusiones del laboratorio	9
6. Laboratorio 2: Análisis dinámico de seguridad de una API	10
6.1. Objetivo y alcance	10
6.2. Herramientas y entorno	10
6.3. Preparación del entorno	10

6.4. Configuración de Burp Suite	11
6.5. Creación de la cuenta y registro del vehículo	11
6.6. Identificación de información expuesta en el foro	13
6.7. Consulta autorizada del vehículo propio	14
6.8. Comprobación de BOLA/IDOR	14
6.9. Hallazgos e impacto técnico	15
6.10. Mitigación recomendada	16
6.11. Conclusiones del laboratorio	16
7. Laboratorio 3: Integración de SAST en un pipeline CI/CD	16
7.1. Objetivo y alcance	16
7.2. Repositorio y aplicación de prueba	17
7.3. Configuración del pipeline de seguridad	17
7.4. Resultados de la ejecución	19
7.5. Hallazgos e impacto técnico	21
7.6. Mitigación recomendada	21
7.7. Conclusiones del laboratorio	21
8. Análisis integral de resultados	21
9. Conclusiones	23

1. Resumen

El presente reporte documenta la ejecución y los resultados de una serie de laboratorios orientados a la evaluación de vulnerabilidades en interfaces de programación de aplicaciones (APIs). Las actividades consideran controles de seguridad en distintas capas del ciclo de vida del software: la configuración perimetral de un API Gateway, el análisis dinámico de una aplicación en ejecución y la incorporación de análisis estático en un pipeline de integración continua.

El trabajo adopta un enfoque DevSecOps con el propósito de identificar riesgos, comprobar sus efectos mediante evidencia experimental y analizar mecanismos de mitigación que puedan integrarse de forma temprana y automatizada al proceso de desarrollo. Para cada laboratorio se presentarán el entorno utilizado, el procedimiento, las evidencias obtenidas, los hallazgos técnicos y las recomendaciones derivadas.

Palabras clave: seguridad de APIs, DevSecOps, SAST, DAST, API Gateway, automatización de seguridad.

2. Introducción

Las APIs constituyen un componente esencial de las arquitecturas modernas, pues permiten la comunicación entre aplicaciones, servicios y plataformas. Esta exposición también amplía la superficie de ataque y hace necesario aplicar controles que abarquen tanto la infraestructura como el código y la lógica de negocio.

DevSecOps propone integrar la seguridad de manera continua durante el ciclo de vida del desarrollo. Bajo este enfoque, las vulnerabilidades no se atienden únicamente al final del proyecto, sino que se buscan, verifican y corrigen desde etapas tempranas mediante prácticas y herramientas automatizadas. Los laboratorios descritos en este reporte permiten observar de forma práctica la relación entre una configuración insegura, su impacto técnico y los controles necesarios para reducir el riesgo.

3. Objetivos

3.1. Objetivo general

Evaluar experimentalmente vulnerabilidades relevantes en APIs y analizar la efectividad de controles preventivos y automatizados bajo un enfoque DevSecOps.

3.2. Objetivos específicos

- Examinar la configuración de seguridad de un API Gateway e identificar políticas perimetrales inseguras.
- Comprobar vulnerabilidades de autorización y lógica de negocio mediante técnicas de análisis dinámico.

- Integrar análisis estático de seguridad en un pipeline de integración continua.
- Documentar los hallazgos mediante resultados reproducibles y capturas de pantalla comentadas.
- Proponer medidas de mitigación acordes con las prácticas de seguridad en el desarrollo de software.

4. Metodología general

El estudio se organiza como una evaluación experimental compuesta por tres laboratorios. En cada uno se establece una línea base, se ejecutan pruebas controladas, se recopilan evidencias y se analiza el efecto de los controles aplicados. La documentación de cada práctica seguirá la misma estructura para facilitar la comparación de resultados:

1. objetivo y alcance del laboratorio;
2. herramientas y entorno de ejecución;
3. procedimiento experimental;
4. resultados y evidencias;
5. hallazgos e impacto técnico;
6. medidas de mitigación; y
7. conclusiones parciales.

Las pruebas se realizan exclusivamente en entornos controlados y con fines académicos. Las credenciales, correos, identificadores y coordenadas visibles en las capturas pertenecen a datos ficticios generados para la demostración y no corresponden a sistemas o personas reales.

5. Laboratorio 1: Evaluación de seguridad de un API Gateway

5.1. Objetivo y alcance

El objetivo del laboratorio fue evaluar la postura de seguridad de un API Gateway configurado como punto único de entrada hacia un servicio de backend. Para ello se desplegó una arquitectura local basada en contenedores, se publicó temporalmente el Gateway mediante un túnel HTTPS y se analizaron las cabeceras HTTP expuestas al cliente con dos servicios externos de evaluación.

El alcance se concentró en controles de seguridad perimetral observables en las respuestas HTTP. Por tanto, las calificaciones obtenidas representan la configuración de cabeceras del Gateway en el momento del escaneo y no constituyen, por sí solas, una evaluación completa de la lógica de negocio o del código fuente de la API.

5.2. Herramientas y entorno de ejecución

El entorno fue desplegado mediante Docker Compose sobre un equipo Windows. La arquitectura estuvo formada por los siguientes componentes:

- **Servicio de backend:** contenedor `backend-api`, construido a partir de la imagen `mendhak/http-https-echo:34` y configurado para escuchar en el puerto interno 8080.
- **API Gateway:** contenedor `nginx-gateway`, basado en `nginx:alpine`, con el puerto 80 del contenedor publicado en el puerto 80 del equipo anfitrión.
- **Configuración del Gateway:** montaje del archivo local `nginx.conf` en `/etc/nginx/conf.d/default.conf`.
- **Exposición temporal:** `ngrok` versión 3.39.9, utilizado para redirigir una dirección HTTPS pública al servicio local `http://localhost:80`.
- **Evaluadores externos:** MDN HTTP Observatory y SecurityHeaders.com.

El backend no publicó directamente ningún puerto al equipo anfitrión. Esta decisión obligó a que las solicitudes externas pasaran por Nginx y permitió que las evaluaciones reflejaran la configuración efectiva del Gateway.

5.3. Procedimiento experimental

5.3.1. Despliegue de los servicios

Se definieron los servicios `web-api` y `api-gateway` en el archivo `docker-compose.yml`. El Gateway declaró una dependencia respecto al backend y cargó su configuración desde el sistema anfitrión. La figura 1 muestra la definición utilizada.

La separación entre el backend y el puerto publicado resulta relevante para la prueba: únicamente el API Gateway quedó accesible desde el exterior. Los servicios se crearon e iniciaron en segundo plano mediante el siguiente comando:

```
docker compose up -d
```

5.3.2. Publicación temporal del API Gateway

Una vez confirmado el funcionamiento local del Gateway en el puerto 80, se inició `ngrok` desde una terminal de Windows:

```
ngrok.exe http 80
```

`Ngrok` generó el dominio temporal `woven-scarf-mantra.ngrok-free.dev` y redirigió el tráfico HTTPS recibido hacia `http://localhost:80`, como se observa en la figura 2. El túnel permitió que los servicios externos de evaluación accedieran al entorno local durante la sesión de prueba.

```
docker-compose.yml X
C: > Users > angel > OneDrive > Escritorio > Estancia Delfin > Laboratorio 1 > docker-compose.yml
1  version: '3.8'
2
3  > Run All Services
4  services:
5      > Run Service
6      web-api:
7          image: mendhak/http-https-echo:34
8          container_name: backend-api
9          environment:
10             - HTTP_PORT=8080
11             # No exponemos los puertos al host para obligar a que todo el tráfico pase por el Gateway
12
13      > Run Service
14      api-gateway:
15          image: nginx:alpine
16          container_name: nginx-gateway
17          ports:
18             - "80:80"
19          volumes:
20             # Enlazaremos el archivo de configuración desde nuestra máquina local al contenedor
21             - ./nginx.conf:/etc/nginx/conf.d/default.conf
22          depends_on:
23             - web-api
```

Figura 1: Definición de los servicios del laboratorio en `docker-compose.yml`.

```
ngrok (Ctrl+C to quit)
♦ One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status      online
Account             MAD312 (Plan: Free)
Version             3.39.9
Region              United States (us)
Web Interface        http://127.0.0.1:4040
Forwarding           https://woven-scarf-mantra.ngrok-free.dev -> http://localhost:80
```

Figura 2: Túnel de ngrok activo y redirección hacia el API Gateway local.

5.3.3. Ejecución de los escaneos

La URL pública se introdujo en MDN HTTP Observatory y en SecurityHeaders.com. Ambos servicios inspeccionaron las cabeceras devueltas por el Gateway y compararon la respuesta con sus respectivos criterios de seguridad. Los análisis corresponden a una línea base previa a la aplicación de medidas de endurecimiento.

Como comprobación complementaria se envió desde Postman una solicitud de preflight `OPTIONS` hacia `/api/`. La petición declaró como origen `http://sitio-atacante.com` y solicitó autorización para utilizar el método `GET`. El servidor devolvió `204 No Content`, tal como se observa en la figura 3.

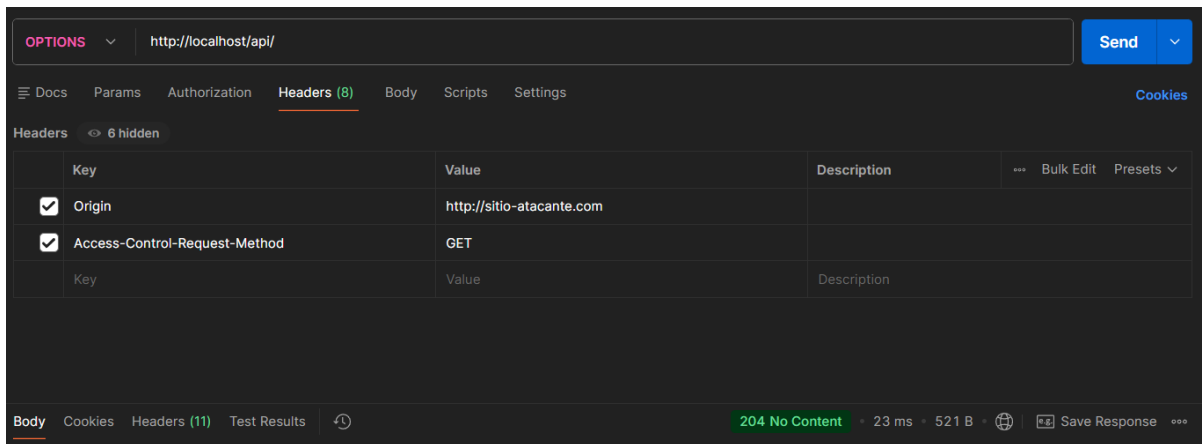


Figura 3: Solicitud preflight enviada desde Postman con un origen externo no confiable.

La captura confirma que el bloque de preflight responde a cualquier solicitud `OPTIONS`. Debido a que la vista no muestra las cabeceras de respuesta, la política `CORS` permisiva se comprueba conjuntamente con la configuración inicial de Nginx presentada posteriormente en la figura 6.

5.4. Resultados y evidencias

5.4.1. Resultado de MDN HTTP Observatory

MDN HTTP Observatory asignó una puntuación de 45 sobre 100, equivalente a una calificación `C-`, con 6 de 10 pruebas superadas. En la evidencia visible también se identifica la ausencia de una política *Content Security Policy* (CSP), condición que produjo una penalización de 25 puntos.

La figura 4 demuestra que el endpoint era accesible desde Internet y que el Gateway satisfacía parte de los controles examinados. Sin embargo, la calificación evidencia que la configuración inicial todavía carecía de mecanismos de protección recomendados para contenido consumido desde navegadores.

5.4.2. Resultado de SecurityHeaders.com

SecurityHeaders.com asignó una calificación `D` al mismo dominio. El reporte, generado el 20 de julio de 2026 a las 03:15:45 UTC, reconoció la presencia de `Referrer-Policy`, pero señaló la ausen-

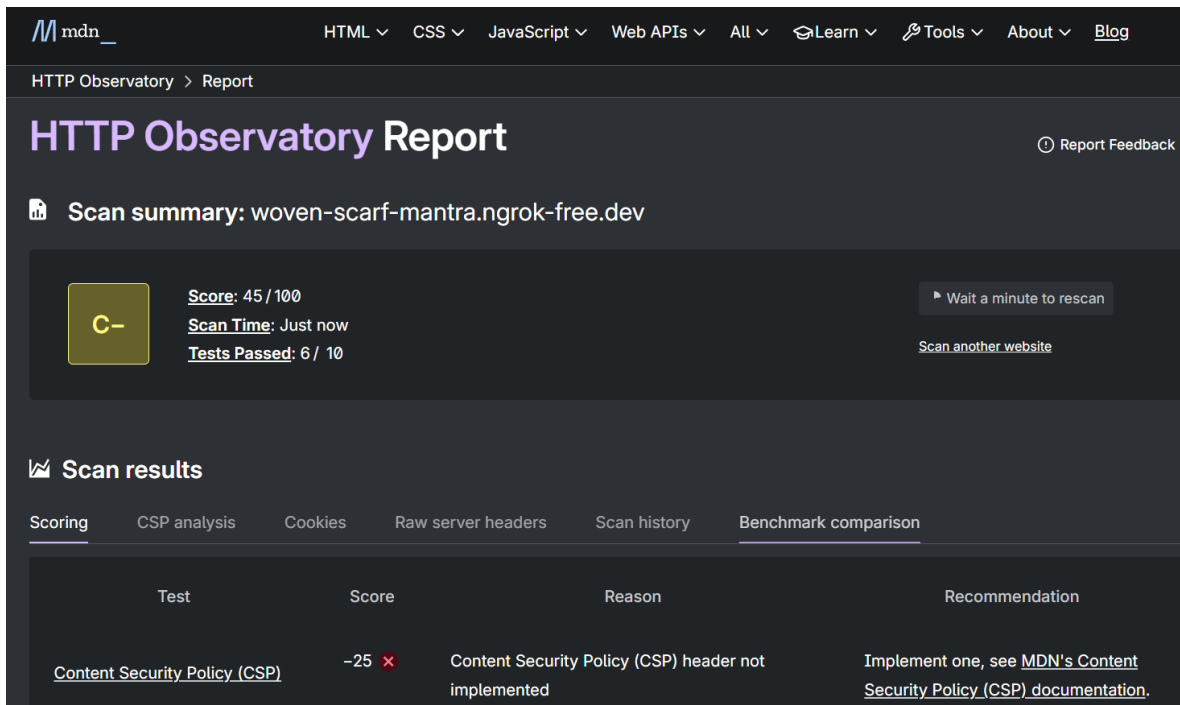


Figura 4: Resultado inicial del análisis realizado con MDN HTTP Observatory.

cia de las cabeceras `Strict-Transport-Security`, `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options` y `Permissions-Policy`.

La coincidencia entre ambos escáneres respecto a la ausencia de CSP incrementa la confianza en el hallazgo. SecurityHeaders.com amplía el diagnóstico al identificar otros controles no incluidos en la respuesta del Gateway.

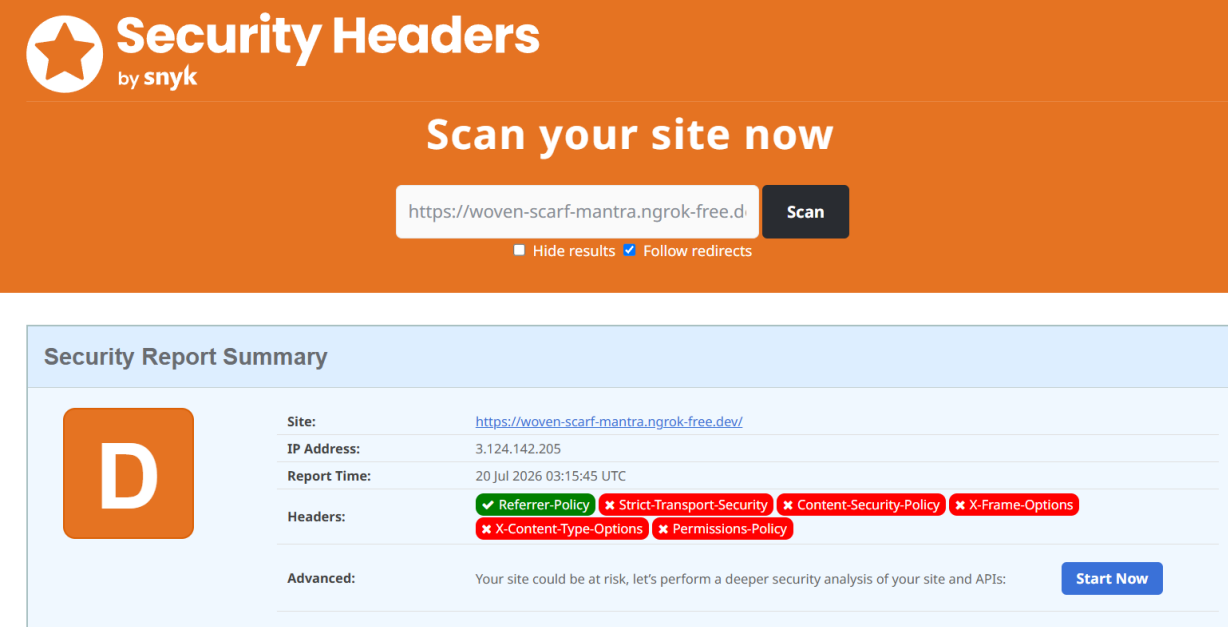
5.5. Hallazgos e impacto técnico

La severidad se asignó de forma contextual. Debido a que el servicio evaluado funciona principalmente como API, algunas cabeceras orientadas a documentos HTML tienen un efecto menor que en una aplicación web tradicional. No obstante, `HSTS` y `X-Content-Type-Options` continúan siendo medidas pertinentes para fortalecer el transporte y el manejo seguro de las respuestas.

5.6. Mitigación aplicada

5.6.1. Configuración inicial vulnerable

La configuración inicial de Nginx permitía solicitudes desde cualquier origen mediante la cabecera `Access-Control-Allow-Origin` con valor `*` y, simultáneamente, habilitaba `Access-Control-Allow-Credentials` con valor `true`. Esta combinación no es válida para solicitudes con credenciales en navegadores conformes al estándar CORS: el comodín no puede emplearse cuando se autoriza el envío de credenciales. Además de ser inconsistente, la política no establecía una lista explícita de



Security Headers
by snyk

Scan your site now

https://woven-scarf-mantra.ngrok-free.d **Scan**

■ Hide results ☒ Follow redirects

Security Report Summary

D

Site: <https://woven-scarf-mantra.ngrok-free.dev/>

IP Address: 3.124.142.205

Report Time: 20 Jul 2026 03:15:45 UTC

Headers:

- ✓ Referrer-Policy
- ✗ Strict-Transport-Security
- ✗ Content-Security-Policy
- ✗ X-Frame-Options
- ✗ X-Content-Type-Options
- ✗ Permissions-Policy

Advanced: Your site could be at risk, let's perform a deeper security analysis of your site and APIs: **Start Now**

Figura 5: Resumen del análisis inicial realizado con SecurityHeaders.com.

Cuadro 1: Registro de hallazgos del Laboratorio 1.

Hallazgo	Evidencia	Severidad	Impacto
Política CORS inconsistente	El archivo inicial combina <code>Access-Control-Allow-Origin</code> con valor <code>*</code> y <code>Access-Control-Allow-Credentials</code> con valor <code>true</code> .	Media–alta	La política es excesivamente permisiva e incompatible con el modelo de credenciales de los navegadores. Puede producir comportamientos inseguros o fallos de interoperabilidad según el cliente utilizado.
CSP ausente	Detectado por MDN HTTP Observatory y SecurityHeaders.com.	Media	No se restringen explícitamente los orígenes desde los que un navegador puede cargar recursos. Su relevancia aumenta si el Gateway entrega contenido HTML.
HSTS ausente	Detectado por SecurityHeaders.com.	Media	El dominio no comunica al navegador la obligación de utilizar exclusivamente HTTPS en conexiones posteriores.
Protección contra MIME sniffing ausente	Falta de <code>X-Content-Type-Options</code> , según SecurityHeaders.com.	Media	Un navegador podría interpretar una respuesta con un tipo distinto al declarado, especialmente si los tipos de contenido no se configuran correctamente.
Protección contra framing ausente	Falta de <code>X-Frame-Options</code> , según SecurityHeaders.com.	Baja–media	Si se sirven interfaces web, estas podrían cargarse dentro de marcos de otros sitios y quedar expuestas a escenarios de <i>clickjacking</i> .
Permissions Policy ausente	Detectado por SecurityHeaders.com.	Baja	No se restringe de forma explícita el uso de determinadas capacidades del navegador; su impacto depende del tipo de contenido servido.

consumidores autorizados.

El Gateway también permitía los métodos GET, POST, OPTIONS y DELETE sin justificar si todos eran necesarios. Finalmente, el archivo no incorporaba las cabeceras defensivas identificadas como ausentes por los escáneres externos. La figura 6 presenta esta línea base.

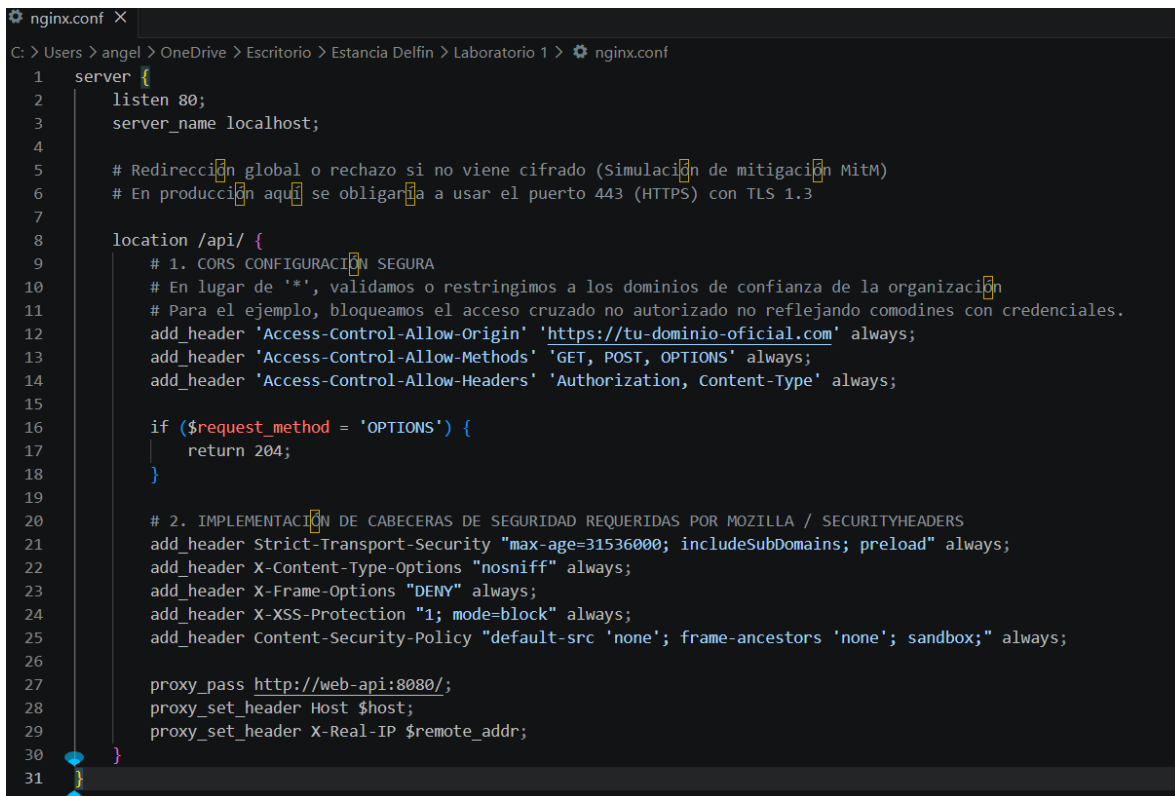
Figura 6: Configuración inicial de Nginx con política CORS permisiva y sin cabeceras de seguridad.

5.6.2. Configuración corregida

La mitigación se implementó directamente en `nginx.conf`, manteniendo el archivo como parte versionable de la infraestructura. Los principales cambios fueron los siguientes:

- sustitución del comodín CORS por el origen autorizado `https://tu-dominio-oficial.com`;
- eliminación de `Access-Control-Allow-Credentials` al no ser necesario para la configuración mostrada;
- reducción de los métodos permitidos a GET, POST y OPTIONS, aplicando el principio de mínimo privilegio;
- incorporación de HSTS con un tiempo máximo de 31,536,000 segundos;
- incorporación de `X-Content-Type-Options: nosniff`;
- bloqueo del uso de marcos mediante `X-Frame-Options: DENY`; y
- definición de una CSP restrictiva con `default-src 'none'`, `frame-ancestors 'none'` y `sandbox`.

La figura 7 muestra la versión endurecida del archivo.



```

1  server {
2      listen 80;
3      server_name localhost;
4
5      # Redirección global o rechazo si no viene cifrado (Simulación de mitigación MitM)
6      # En producción aquí se obligaría a usar el puerto 443 (HTTPS) con TLS 1.3
7
8      location /api/ {
9          # 1. CORS CONFIGURACIÓN SEGURA
10         # En lugar de '*', validamos o restringimos a los dominios de confianza de la organización
11         # Para el ejemplo, bloqueamos el acceso cruzado no autorizado no reflejando comodines con credenciales.
12         add_header 'Access-Control-Allow-Origin' 'https://tu-dominio-oficial.com' always;
13         add_header 'Access-Control-Allow-Methods' 'GET, POST, OPTIONS' always;
14         add_header 'Access-Control-Allow-Headers' 'Authorization, Content-Type' always;
15
16         if ($request_method = 'OPTIONS') {
17             return 204;
18         }
19
20         # 2. IMPLEMENTACIÓN DE CABECERAS DE SEGURIDAD REQUERIDAS POR MOZILLA / SECURITYHEADERS
21         add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload" always;
22         add_header X-Content-Type-Options "nosniff" always;
23         add_header X-Frame-Options "DENY" always;
24         add_header X-XSS-Protection "1; mode=block" always;
25         add_header Content-Security-Policy "default-src 'none'; frame-ancestors 'none'; sandbox;" always;
26
27         proxy_pass http://web-api:8080/;
28         proxy_set_header Host $host;
29         proxy_set_header X-Real-IP $remote_addr;
30     }
31 }

```

Figura 7: Configuración de Nginx después de aplicar restricciones CORS y cabeceras defensivas.

La directiva `X-XSS-Protection` incluida en la configuración se conserva como mecanismo de compatibilidad con navegadores antiguos, pero no debe considerarse un control principal, ya que los navegadores modernos han dejado de implementarla. La protección frente a inyección de contenido debe apoyarse principalmente en CSP, validación de entradas, codificación de salidas y tipos de contenido correctos.

Asimismo, los parámetros `includeSubDomains` y `preload` de HSTS deben utilizarse únicamente cuando el servicio opere bajo un dominio propio, todos sus subdominios admitan HTTPS y se hayan verificado los requisitos del programa de precarga. En el dominio temporal de ngrok su presencia tiene un propósito demostrativo y no implica que dicho dominio pueda registrarse para precarga.

La configuración corregida atiende los hallazgos principales de la línea base, aunque todavía sería conveniente declarar una `Permissions-Policy` acorde con el servicio. También se recomienda reemplazar el dominio ilustrativo por el origen real autorizado antes de desplegar el Gateway en un entorno operativo.

5.7. Conclusiones del laboratorio

El laboratorio demostró que una API funcional y accesible mediante HTTPS puede conservar una postura perimetral incompleta. MDN HTTP Observatory y SecurityHeaders.com produjeron calificaciones diferentes debido a sus criterios de evaluación, pero coincidieron en la ausencia de

controles relevantes, particularmente CSP. El análisis del archivo inicial permitió identificar, además, una política CORS inconsistente y una superficie de métodos mayor que la necesaria.

La versión corregida de `nginx.conf` restringe el origen permitido, reduce los métodos expuestos e incorpora las principales cabeceras defensivas señaladas por los escáneres. De esta manera, la configuración como código proporciona una mitigación reproducible y susceptible de revisión dentro de un flujo DevSecOps.

Las calificaciones C– y D documentan la línea base previa al endurecimiento. Debido a que no se dispone de capturas de un segundo escaneo, el cierre del laboratorio confirma la aplicación de los controles a nivel de configuración, pero no atribuye una nueva calificación externa al estado corregido. En una iteración futura, el re-escaneo automatizado debería utilizarse como criterio de aceptación antes del despliegue.

6. Laboratorio 2: Análisis dinámico de seguridad de una API

6.1. Objetivo y alcance

El objetivo de este laboratorio fue evaluar dinámicamente los controles de autorización de una API en ejecución. En particular, se comprobó si un usuario autenticado podía acceder a la ubicación de un vehículo ajeno mediante la sustitución del identificador incluido en la ruta de una petición HTTP.

Las pruebas se realizaron exclusivamente en OWASP crAPI (*completely Ridiculous API*), un entorno local deliberadamente vulnerable y destinado al aprendizaje de seguridad en APIs. Los correos, tokens, cookies, vehículos y coordenadas que aparecen en las evidencias son datos ficticios generados dentro del laboratorio; no corresponden a personas, cuentas ni sistemas reales.

6.2. Herramientas y entorno

- repositorio público OWASP/crAPI alojado en GitHub;
- Docker y Docker Compose para desplegar los microservicios;
- Burp Suite Community Edition v2026.4.3 como proxy de análisis dinámico;
- aplicación crAPI en <http://127.0.0.1:8888>;
- MailHog en <http://localhost:8025> para consultar los mensajes generados por el laboratorio.

6.3. Preparación del entorno

El código se obtuvo del repositorio oficial <https://github.com/OWASP/crAPI>, mostrado en la figura 8. Dentro del proyecto se utilizó el subdirectorio relativo `deploy/docker`, que contiene el archivo de composición necesario para iniciar la aplicación.

Primero se descargaron las imágenes de los servicios:

```
docker compose pull
```

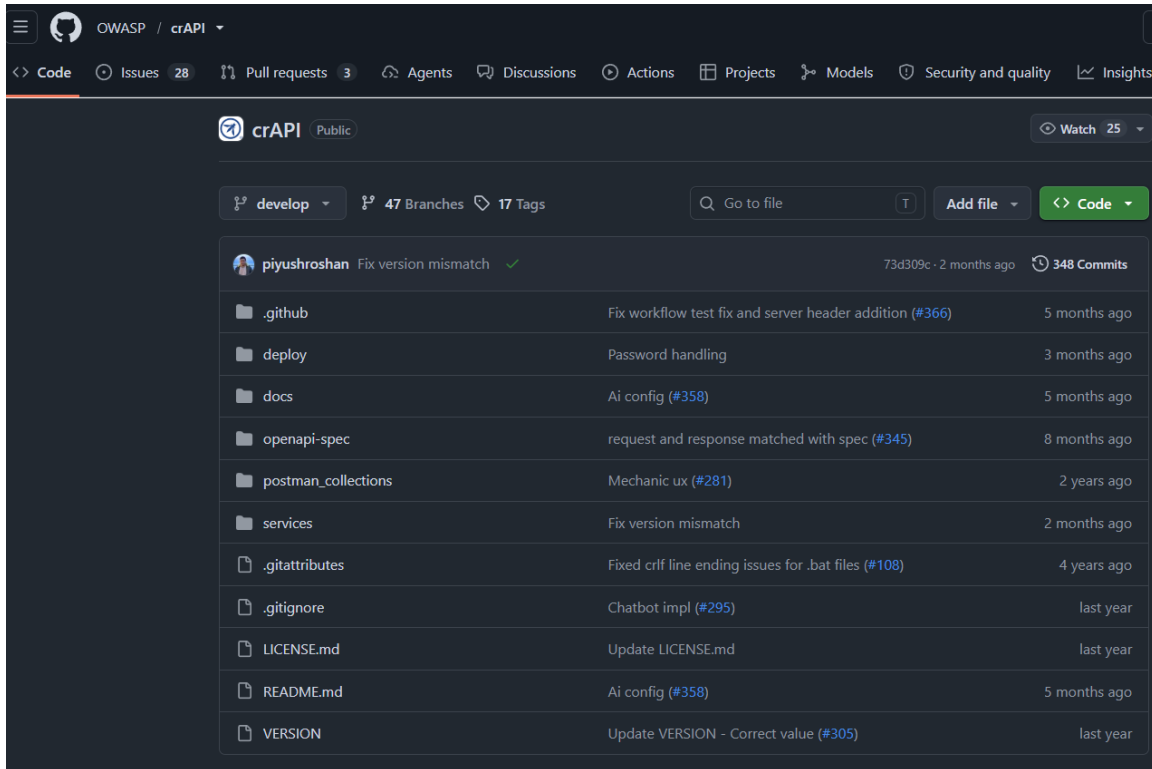


Figura 8: Repositorio de OWASP crAPI empleado para el segundo laboratorio.

Después se inició el entorno en segundo plano utilizando el modo de compatibilidad:

```
docker compose -f docker-compose.yml --compatibility up -d
```

6.4. Configuración de Burp Suite

Burp Suite se utilizó como intermediario entre el navegador y crAPI. Desde el módulo *Proxy* se abrió el navegador integrado y se registraron las transacciones en *HTTP history*. Posteriormente, las peticiones relevantes se enviaron a *Repeater* para modificar parámetros y repetirlas de forma controlada.

6.5. Creación de la cuenta y registro del vehículo

Se creó una cuenta controlada y se inició sesión en la aplicación, como se observa en la figura 10. Esta cuenta proporcionó credenciales válidas para establecer una línea base de acceso autorizado.

El proceso de registro generó en MailHog un mensaje con un VIN y un código PIN sintéticos. Estos datos fueron utilizados para asociar un vehículo a la cuenta de prueba.

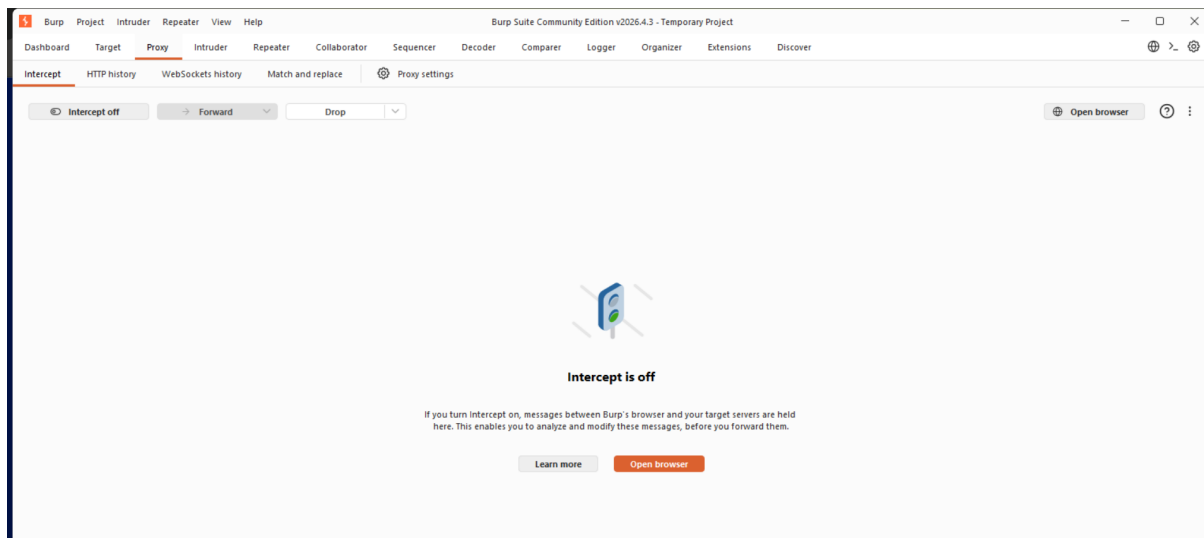


Figura 9: Módulo Proxy de Burp Suite utilizado para registrar el tráfico de crAPI.

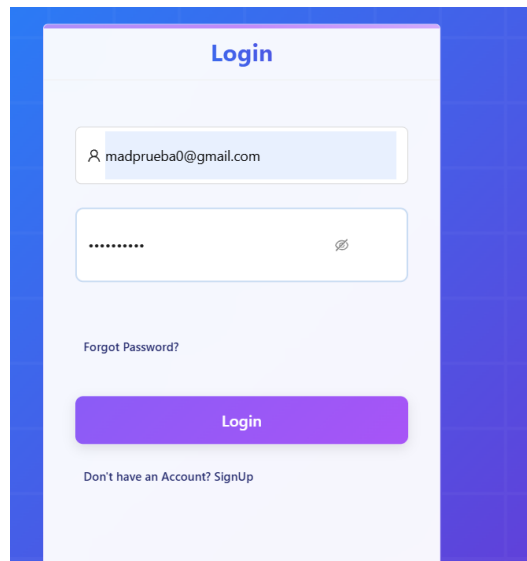


Figura 10: Inicio de sesión con la cuenta controlada del laboratorio.

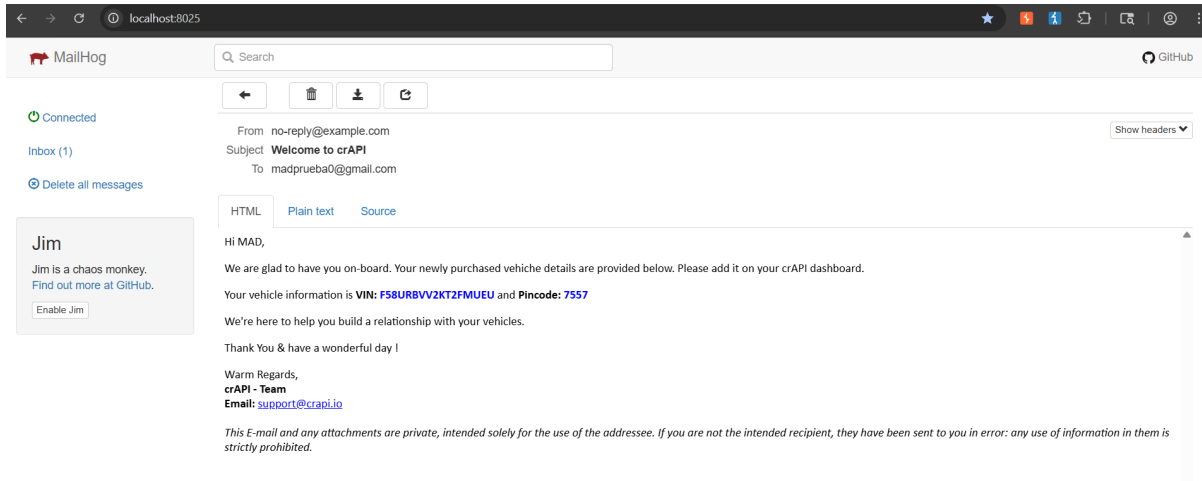


Figura 11: VIN y código PIN recibidos en MailHog para registrar el vehículo de prueba.

6.6. Identificación de información expuesta en el foro

Al visitar <http://127.0.0.1:8888/forum>, el navegador generó una petición al endpoint de publicaciones recientes:

```
/community/api/v2/community/posts/recent?limit=30&offset=0
```

La respuesta JSON incluyó propiedades internas de los autores, entre ellas el correo electrónico y el campo `vehicleid`.

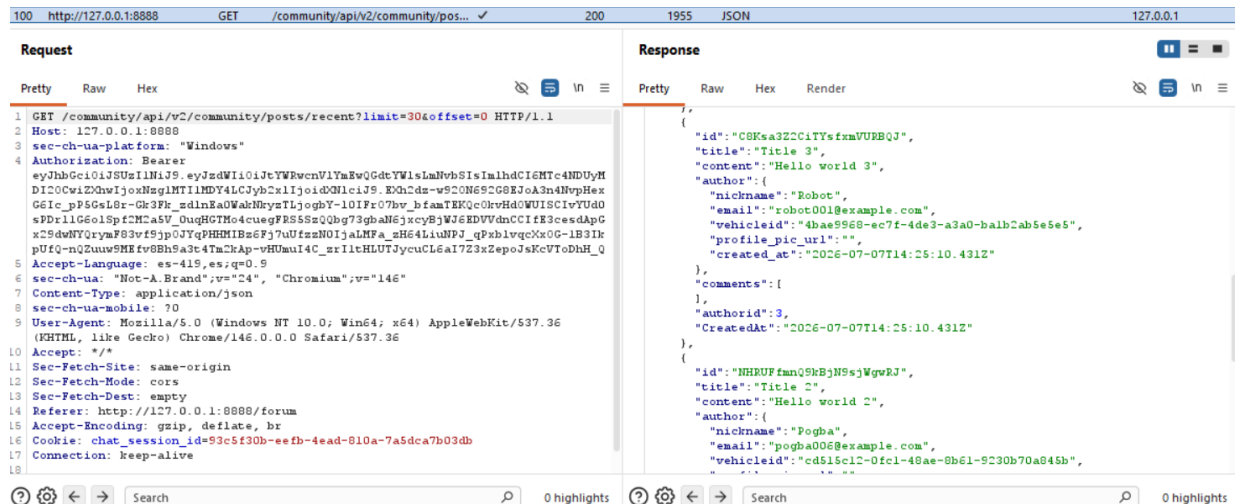


Figura 12: Respuesta del foro que expone correos e identificadores de vehículos de otros usuarios.

La información expuesta no era necesaria para representar públicamente los comentarios. En particular, el identificador del vehículo proporcionó el dato requerido para intentar acceder posteriormente a un recurso perteneciente a otro usuario.

6.7. Consulta autorizada del vehículo propio

En el historial de Burp Suite se identificó la petición empleada por la aplicación para obtener la ubicación del vehículo asociado a la cuenta autenticada:

```
GET /identity/api/v2/vehicle/{carId}/location
```

La solicitud contenía un token válido y el identificador del vehículo propio. El servidor devolvió 200 OK, junto con el `carId`, las coordenadas y los datos de la cuenta correspondiente.

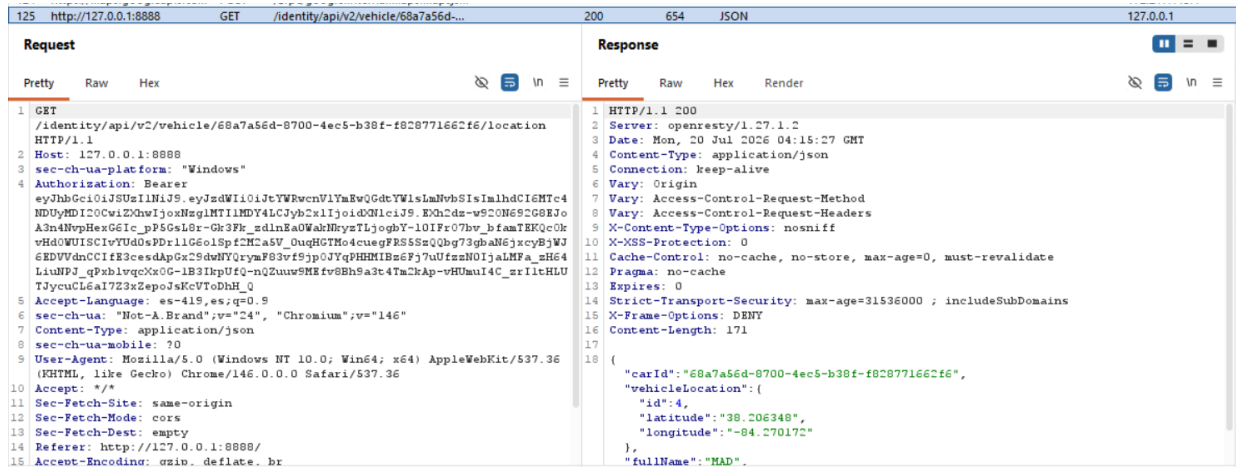


Figura 13: Petición legítima para consultar la ubicación del vehículo asociado al usuario autenticado.

La petición se trasladó a *Repeater* y se ejecutó nuevamente sin alterar el token ni el identificador. Esta repetición estableció la respuesta autorizada que se utilizaría como línea base para la prueba.

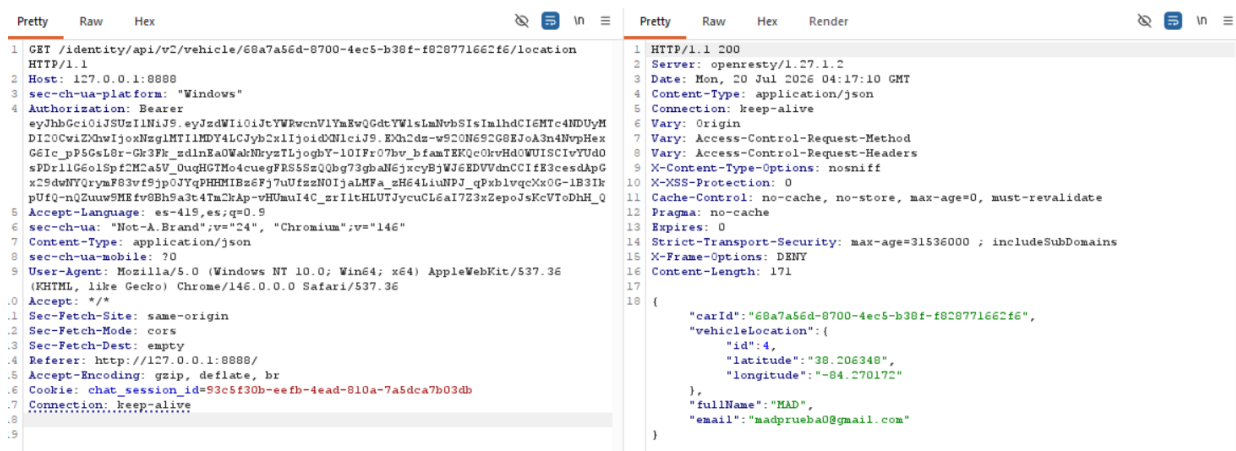


Figura 14: Repetición controlada de la consulta correspondiente al vehículo propio.

6.8. Comprobación de BOLA/IDOR

Para comprobar la autorización a nivel de objeto, se mantuvo el mismo token de la cuenta controlada y se substituyó únicamente el `carId` de la ruta por un identificador perteneciente a otro usuario,

obtenido previamente en la respuesta del foro.

El servidor aceptó la petición modificada y respondió nuevamente con 200 OK. La respuesta reveló el nombre, correo y coordenadas del vehículo ajeno, como se muestra en la figura 15.

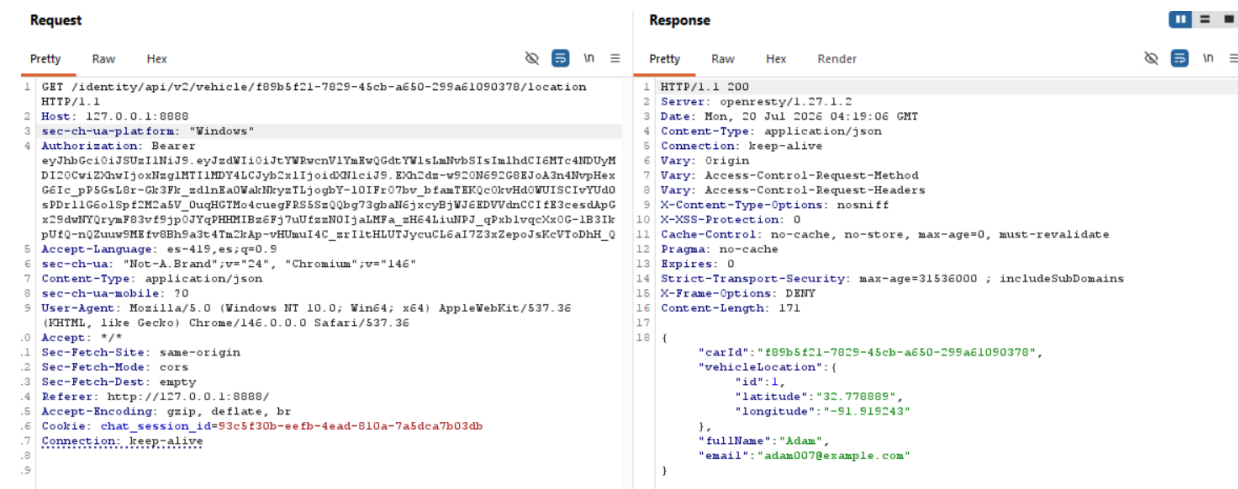


Figura 15: Acceso no autorizado a la ubicación de otro vehículo mediante la sustitución del `carId`.

El resultado demuestra que el servicio verificó la autenticación, ya que exigió un token válido, pero no comprobó que el vehículo solicitado perteneciera al sujeto representado por dicho token. El comportamiento corresponde a una vulnerabilidad de autorización a nivel de objeto (BOLA), también descrita en este escenario como IDOR, y permite una escalación horizontal de privilegios.

6.9. Hallazgos e impacto técnico

Cuadro 2: Hallazgos principales del Laboratorio 2.

Hallazgo	Evidencia	Severidad	Impacto
Exposición de propiedades internas	El servicio del foro devuelve correos e identificadores de vehículos de otros usuarios.	Media	Facilita la enumeración de recursos y el encadenamiento con otras vulnerabilidades.
BOLA/IDOR en la consulta de ubicación	Cambiar el <code>carId</code> sin modificar el token produce 200 OK y devuelve información de otro propietario.	Alta	Expone datos personales y coordenadas de geolocalización mediante escalación horizontal.
Falta de vinculación sujeto-objeto	El backend autentica al solicitante, pero no comprueba la propiedad del vehículo.	Alta	Cualquier identificador válido conocido podría utilizarse para consultar un recurso ajeno.

El atributo afectado principalmente es la confidencialidad. El uso de identificadores UUID dificulta la adivinación aleatoria, pero no sustituye un control de autorización. En este caso, la propia aplicación divulgó los identificadores necesarios para realizar la prueba.

6.10. Mitigación recomendada

El servicio debe obtener la identidad del usuario a partir del token validado y comprobar, antes de consultar la ubicación, que el vehículo solicitado pertenece a ese usuario. Cuando no exista dicha relación, la API debe finalizar sin devolver información mediante una respuesta `403 Forbidden` o `404 Not Found`, según la política adoptada.

También se recomienda:

- eliminar correos, `vehicleid` y propiedades internas de las respuestas públicas del foro;
- utilizar objetos de transferencia específicos para evitar serializar entidades completas;
- incorporar pruebas negativas que intenten acceder a recursos de un segundo usuario;
- ejecutar automáticamente las pruebas de autorización en el pipeline CI/CD; y
- revisar todos los endpoints que acepten identificadores de objetos.

6.11. Conclusiones del laboratorio

El laboratorio demostró que una autenticación válida no garantiza una autorización correcta. Manteniendo las credenciales de una cuenta ordinaria y modificando solamente un identificador de la ruta fue posible consultar la ubicación de un vehículo ajeno. La respuesta `200 OK` y la devolución de coordenadas de otro propietario constituyen evidencia directa de BOLA.

La prueba también mostró cómo dos debilidades pueden encadenarse: la exposición del identificador en el foro facilitó la explotación de la ausencia de control de propiedad en el servicio de identidad. Desde el enfoque DevSecOps, esta clase de defecto debe prevenirse mediante pruebas automatizadas de autorización tanto positivas como negativas.

7. Laboratorio 3: Integración de SAST en un pipeline CI/CD

7.1. Objetivo y alcance

El objetivo del tercer laboratorio fue demostrar la incorporación de controles de seguridad automatizados dentro de un flujo de integración continua. Para ello se creó una aplicación Python intencionalmente vulnerable y se configuró un workflow de GitHub Actions encargado de ejecutar Semgrep después de cada actualización relevante del repositorio.

La práctica se concentró en el análisis estático de seguridad (SAST), es decir, en la inspección del código sin necesidad de ejecutar la aplicación. El criterio de aceptación consistió en impedir que el pipeline finalizara satisfactoriamente cuando el escáner detectara patrones inseguros.

7.2. Repositorio y aplicación de prueba

La demostración se implementó en el repositorio público MADH312/devsecops-sast-lab. La rama principal contiene el archivo `app.py`, un archivo de documentación y el directorio `.github/workflows`, donde se almacenó la definición del pipeline.

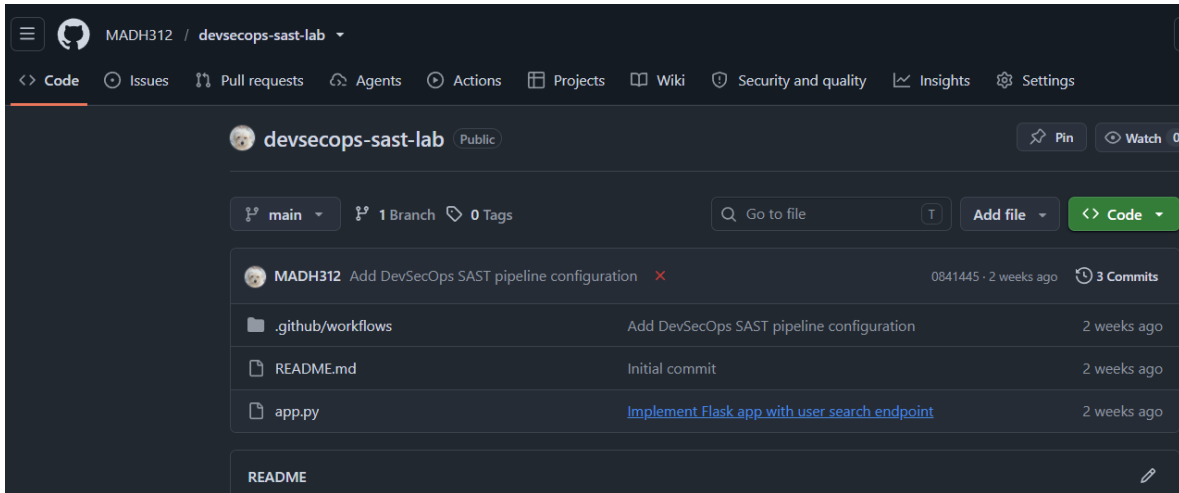


Figura 16: Repositorio de GitHub utilizado para demostrar la integración de SAST.

La aplicación de prueba fue desarrollada con Flask y SQLite. En el código se introdujeron deliberadamente dos debilidades principales, visibles en la figura 17:

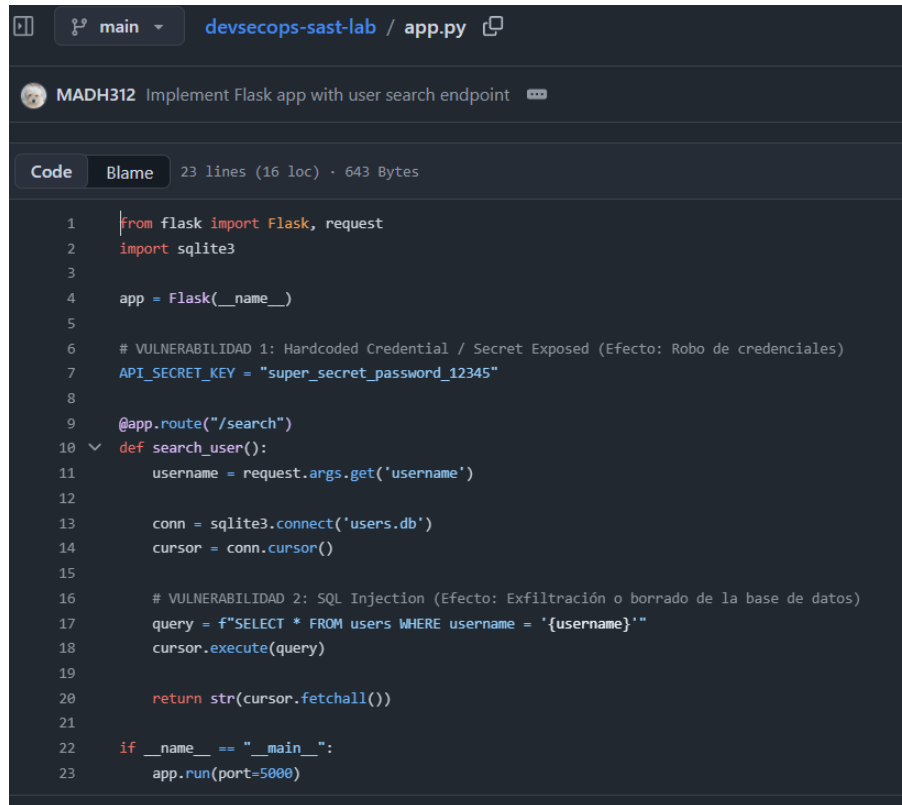
- una clave secreta almacenada directamente en el código fuente mediante la variable `API_SECRET_KEY`; y
- una consulta SQL construida con una cadena formateada que incorpora directamente el parámetro `username` recibido desde la petición.

El secreto escrito en texto plano podría quedar expuesto a cualquier persona con acceso al repositorio o a su historial. Por otra parte, la concatenación de la entrada dentro de la sentencia SQL permitiría alterar la estructura de la consulta, acceder a registros no autorizados o modificar información si la cuenta de base de datos dispusiera de permisos suficientes.

7.3. Configuración del pipeline de seguridad

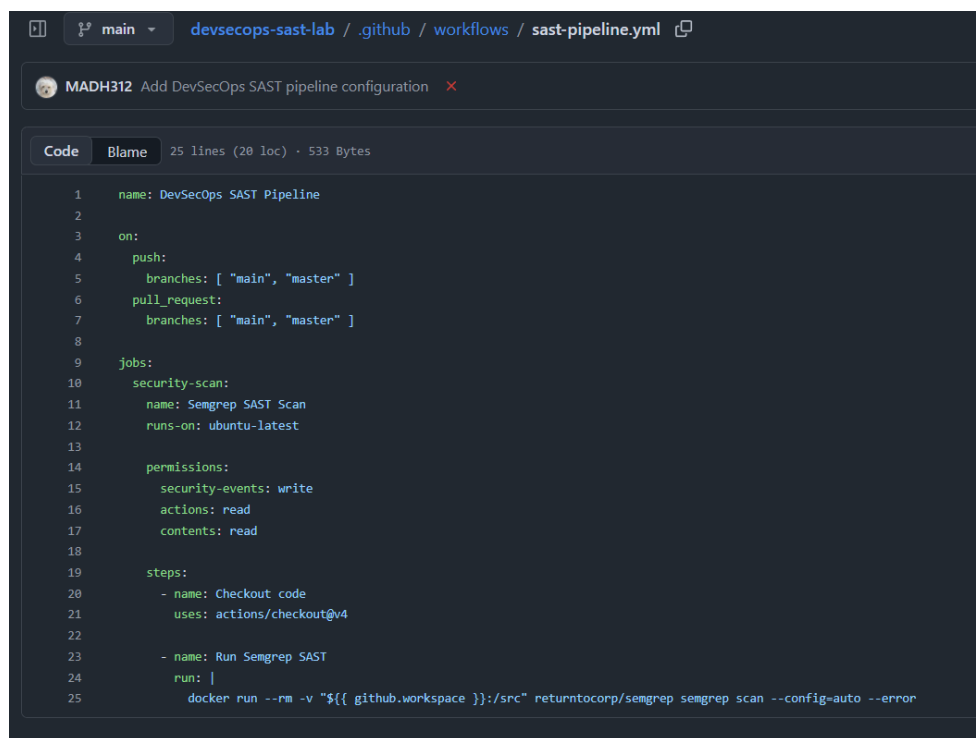
El workflow se almacenó en `.github/workflows/sast-pipeline.yml`. La configuración declaró como eventos de activación los `push` y las solicitudes de cambio dirigidas a las ramas `main` y `master`. De este modo, el análisis se ejecuta cuando se publica código nuevo o cuando se propone su incorporación a la rama principal.

El job `security-scan` utiliza un agente `ubuntu-latest`. Primero recupera el contenido del repositorio con `actions/checkout@v4`; posteriormente inicia la imagen Docker de Semgrep, monta el espacio de trabajo en el directorio `/src` del contenedor y ejecuta el siguiente análisis:



```
1  from flask import Flask, request
2  import sqlite3
3
4  app = Flask(__name__)
5
6  # VULNERABILIDAD 1: Hardcoded Credential / Secret Exposed (Efecto: Robo de credenciales)
7  API_SECRET_KEY = "super_secret_password_12345"
8
9  @app.route("/search")
10 def search_user():
11     username = request.args.get('username')
12
13     conn = sqlite3.connect('users.db')
14     cursor = conn.cursor()
15
16     # VULNERABILIDAD 2: SQL Injection (Efecto: Exfiltración o borrado de la base de datos)
17     query = f"SELECT * FROM users WHERE username = '{username}'"
18     cursor.execute(query)
19
20     return str(cursor.fetchall())
21
22 if __name__ == "__main__":
23     app.run(port=5000)
```

Figura 17: Aplicación Python con un secreto embebido y una consulta vulnerable a inyección SQL.



```
1  name: DevSecOps SAST Pipeline
2
3  on:
4    push:
5      branches: [ "main", "master" ]
6    pull_request:
7      branches: [ "main", "master" ]
8
9  jobs:
10   security-scan:
11     name: Semgrep SAST Scan
12     runs-on: ubuntu-latest
13
14     permissions:
15       security-events: write
16       actions: read
17       contents: read
18
19     steps:
20       - name: Checkout code
21         uses: actions/checkout@v4
22
23       - name: Run Semgrep SAST
24         run: |
25           docker run --rm -v "${{ github.workspace }}" :/src returntocorp/semgrep semgrep scan --config=auto --error
```

Figura 18: Workflow de GitHub Actions configurado para ejecutar Semgrep.

```
semgrep scan --config=auto --error
```

La opción `-config=auto` permite seleccionar reglas pertinentes a las tecnologías detectadas. La opción `-error` convierte los hallazgos en un código de salida distinto de cero, por lo que el job falla y funciona como un *quality gate* de seguridad.

7.4. Resultados de la ejecución

Después de publicar la configuración en la rama principal, GitHub Actions inició automáticamente el workflow. La ejecución terminó con estado *Failure*: el proceso completo duró 26 segundos y el paso de Semgrep empleó 23 segundos. El código de salida fue 1, comportamiento esperado al haberse identificado vulnerabilidades bajo la política configurada.

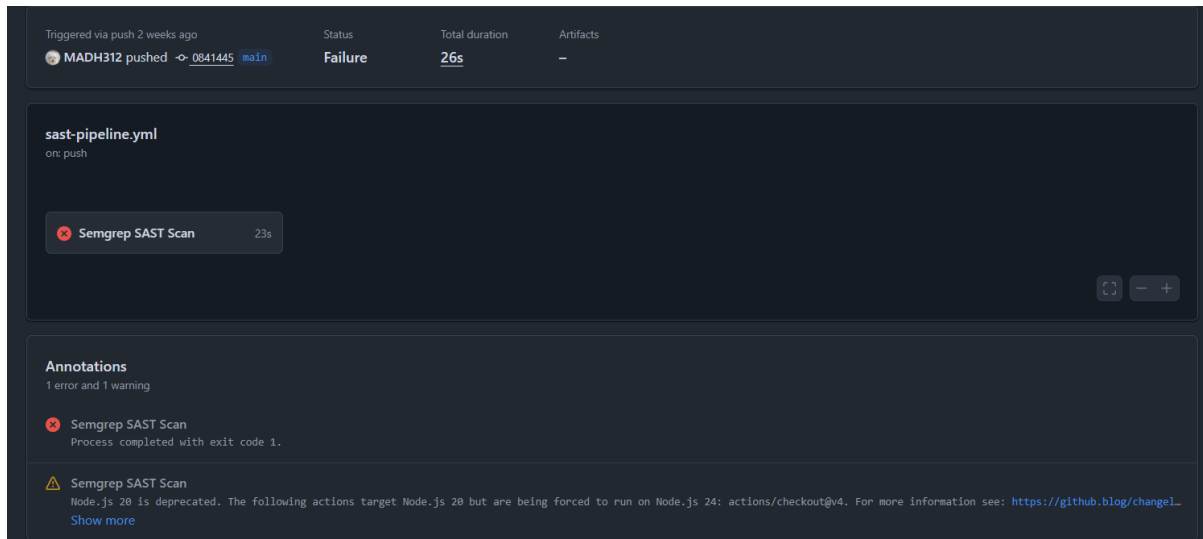


Figura 19: Ejecución bloqueada por el quality gate de Semgrep en GitHub Actions.

La advertencia relacionada con la versión de Node.js utilizada por `actions/checkout` pertenece al entorno de ejecución de GitHub Actions y no fue la causa del bloqueo. El fallo se produjo porque el comando de Semgrep devolvió el código de salida 1.

El registro detallado informó seis hallazgos de código. Entre los resultados visibles se encuentran una consulta SQL construida con datos controlados por el usuario y el uso de una etiqueta mutable en `actions/checkout@v4`. Semgrep clasificó ambos resultados mostrados como bloqueantes.

El hallazgo sobre la acción de GitHub advierte que una etiqueta como `@v4` puede cambiar de referencia. Para reducir el riesgo de cadena de suministro, una configuración de alta seguridad puede fijar la acción a un hash de commit completo y previamente verificado. El segundo hallazgo confirma el flujo de datos desde `request.args.get('username')` hasta `cursor.execute(query)`, lo que establece una ruta de entrada controlada por el usuario hacia el intérprete SQL.

```

Semgrep SAST Scan
failed 2 weeks ago in 23s

Run Semgrep SAST

124 | 6 Code Findings |
125 |
126 |
127 | .github/workflows/sast-pipeline.yml
128 | >> yml.github-actions.security.github-actions-mutable-action-tag.github-actions-mutable-action-tag
129 |
130 | « Blocking »
131 | GitHub Actions step uses a mutable tag or branch reference. Tags and branch names can be silently
132 | repointed by the action owner, enabling supply-chain attacks – as seen in the trivy-action and kics-
133 | github-action compromises. Pin the reference to a full 40-character commit SHA instead, e.g. `uses:
134 | actions/checkout@8ade135a41bc03ea155e62e844d188df1ea18608`.
135 | Details: https://sg.run/2LgAL
136 |
137 | 21| uses: actions/checkout@v4
138 |
139 | app.py
140 | >> python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute
141 |
142 | « Blocking »
143 | User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and
144 | therefore protected information could be leaked. Instead, use django's QuerySets, which are built
145 | with query parameterization and therefore not vulnerable to sql injection. For example, you could
146 | use `Entry.objects.filter(date=2006)`.
147 | Details: https://sg.run/qx7y
148 |
149 | 11| username = request.args.get('username')
150 | 12|
151 | 13| conn = sqlite3.connect('users.db')
152 | 14| cursor = conn.cursor()
153 | 15|
154 | 16| # VULNERABILIDAD 2: SQL Injection (Efecto: Exfiltración o borrado de la base de datos)
155 | 17| query = f"SELECT * FROM users WHERE username = '{username}'"
156 | 18| cursor.execute(query)

```

Figura 20: Registro de Semgrep con hallazgos bloqueantes en el workflow y en `app.py`.

Cuadro 3: Hallazgos principales del Laboratorio 3.

Hallazgo	Evidencia	Severidad	Impacto
Secreto embebido	API_SECRET_KEY contiene una credencial directamente en <code>app.py</code> .	Alta	La credencial queda expuesta en el repositorio y en su historial, lo que puede facilitar accesos no autorizados.
Inyección SQL	La entrada <code>username</code> se concatena en una consulta ejecutada mediante <code>cursor.execute</code> .	Alta	Permite alterar consultas y comprometer la confidencialidad o integridad de la base de datos.
Acción con referencia mutable	El workflow utiliza <code>actions/checkout@v4</code> en lugar de un hash de commit completo.	Media	Un cambio malicioso o comprometido en la referencia podría afectar la cadena de suministro del pipeline.
Quality gate activo	GitHub Actions finalizó con estado <i>Failure</i> y código de salida 1.	Control	Impide que el flujo inseguro sea considerado satisfactorio y proporciona retroalimentación inmediata.

7.5. Hallazgos e impacto técnico

7.6. Mitigación recomendada

La credencial debe eliminarse del código y sustituirse por una variable de entorno o por un secreto administrado mediante GitHub Actions Secrets. Debido a que un secreto confirmado en el historial debe considerarse comprometido, también sería necesario revocarlo, generar uno nuevo y revisar los commits previos.

La consulta SQL debe parametrizarse para separar las instrucciones de los valores proporcionados por el usuario. En SQLite, una forma segura sería:

```
cursor.execute(  
    "SELECT * FROM users WHERE username = ?",  
    (username,) )
```

Para fortalecer el propio pipeline se recomienda fijar las acciones de terceros a un hash de commit verificado, limitar sus permisos al mínimo necesario y actualizar periódicamente las referencias mediante un proceso controlado. Después de corregir el código, el workflow debe ejecutarse nuevamente para comprobar que el quality gate permita continuar únicamente cuando los hallazgos bloqueantes hayan sido eliminados o formalmente aceptados.

7.7. Conclusiones del laboratorio

El laboratorio demostró la integración efectiva de un control SAST dentro de GitHub Actions. La ejecución automática de Semgrep detectó patrones inseguros, generó evidencia detallada y provocó el fallo del pipeline en menos de un minuto. Esta respuesta inmediata reduce el tiempo entre la introducción de una vulnerabilidad y su identificación.

El resultado representa de forma práctica el principio *shift-left*: la seguridad se evalúa en el mismo flujo donde se integra el código, antes de su promoción a un entorno operativo. A diferencia de una revisión manual aislada, el quality gate aplica una política repetible a cada actualización y convierte los hallazgos en una condición verificable del proceso de entrega.

8. Análisis integral de resultados

Los laboratorios evaluaron la seguridad de las APIs desde tres perspectivas complementarias: la configuración perimetral de la infraestructura, el comportamiento de la aplicación durante su ejecución y la inspección automatizada del código fuente. Los resultados demuestran que ninguna de estas capas proporciona, de manera aislada, cobertura suficiente frente a todos los riesgos observados.

En el Laboratorio 1, los escáneres externos asignaron calificaciones C– y D a la configuración inicial del API Gateway. Los hallazgos se concentraron en una política CORS inconsistente y en la ausencia de cabeceras como HSTS, CSP, X-Content-Type-Options y X-Frame-Options. La modificación de `nginx.conf` permitió aplicar restricciones de origen, reducir los métodos expuestos e incorporar

controles defensivos. Este resultado confirma que la infraestructura como código facilita implementar medidas repetibles y sujetas a revisión.

Sin embargo, el endurecimiento del Gateway no habría impedido la vulnerabilidad identificada en el Laboratorio 2. La prueba dinámica demostró que un usuario autenticado podía sustituir el identificador de su vehículo por el de otro usuario y obtener una respuesta 200 OK con coordenadas ajenas. La debilidad residía en la lógica de autorización del microservicio: el sistema validaba la identidad del solicitante, pero no su relación de propiedad con el recurso. Por tanto, los controles de transporte y cabeceras no sustituyen las verificaciones de autorización dentro de la aplicación.

En el Laboratorio 3 se incorporó Semgrep a un flujo de GitHub Actions. El análisis estático detectó patrones inseguros en la aplicación y en el propio workflow, entre ellos una consulta SQL construida mediante concatenación de entradas y una referencia mutable a una acción. Al configurarse el escáner para devolver un estado de error, el pipeline bloqueó la ejecución insegura en 26 segundos. Este comportamiento materializa el principio *shift-left*: los defectos son identificados antes de que el código avance hacia una rama o entorno de producción.

Cuadro 4: Comparación integral de los controles evaluados.

Laboratorio	Capa evaluada	Aporte principal	Limitación observada
API Gateway	Infraestructura y perímetro	Centraliza CORS, cabeceras de seguridad, enrutamiento y políticas de exposición.	No conoce necesariamente la propiedad de los objetos ni corrige fallos de lógica de negocio.
DAST con crAPI	Aplicación en ejecución	Comprueba el comportamiento real de autenticación, autorización y respuestas de la API.	Detecta el fallo cuando la aplicación ya se encuentra desplegada en un entorno ejecutable.
SAST con Semgrep	Código y pipeline CI/CD	Identifica patrones inseguros tempranamente y puede bloquear cambios que incumplan el umbral de seguridad.	No reproduce por sí solo el contexto completo de ejecución ni todas las reglas de negocio.

La relación entre los hallazgos también evidencia la posibilidad de encadenar debilidades. En crAPI, la respuesta del foro expuso identificadores internos de vehículos; posteriormente, esos valores fueron utilizados para explotar la ausencia de autorización a nivel de objeto. Aunque la exposición inicial tenía una severidad menor, aumentó considerablemente el impacto al combinarse con BOLA. Esto demuestra que la priorización no debe considerar cada hallazgo de forma completamente independiente.

Desde una perspectiva de cobertura, el Gateway actúa como control preventivo común para el tráfico; DAST valida externamente el comportamiento observable de los servicios; y SAST anticipa defectos en el código antes del despliegue. La integración de las tres técnicas establece una estrategia de defensa en profundidad: prevenir configuraciones débiles, comprobar escenarios de abuso y detener cambios inseguros durante la integración continua.

Para que esta estrategia sea medible, los resultados deben convertirse en criterios de aceptación del pipeline. Entre los posibles umbrales se encuentran la ausencia de secretos y patrones de inyección de severidad alta, la ejecución satisfactoria de pruebas negativas de autorización, y la verificación automática de las cabeceras obligatorias del Gateway. Un incumplimiento debería detener la promoción del artefacto hasta que exista una corrección o una excepción formalmente justificada.

9. Conclusiones

La evaluación experimental confirma que la seguridad de una API no puede depender de un único producto o control. Las vulnerabilidades observadas se originaron en capas distintas y requirieron mecanismos de detección diferentes. El API Gateway permitió atender debilidades de configuración perimetral, pero no pudo prevenir una decisión incorrecta de autorización dentro de un microservicio.

El hallazgo de mayor impacto práctico fue BOLA, debido a que una cuenta autenticada pudo consultar la ubicación de un vehículo perteneciente a otro usuario. La prueba evidenció la diferencia fundamental entre autenticación y autorización: comprobar la identidad del solicitante no es suficiente si cada operación omite validar su permiso sobre el objeto solicitado. La mitigación debe aplicarse en el servidor mediante una relación explícita entre el usuario autenticado y el recurso.

También se comprobó que la minimización de datos forma parte de la defensa. La exposición de correos e identificadores de vehículos en el servicio del foro proporcionó información útil para encadenar la explotación. Las respuestas de una API deben incluir únicamente las propiedades necesarias para su consumidor y evitar la serialización directa de entidades internas.

Los resultados atribuidos al análisis estático muestran el valor de incorporar seguridad desde las primeras etapas del desarrollo. La detección de secretos escritos en el código y consultas SQL inseguras, acompañada del fallo automático del pipeline, evita que defectos conocidos avancen silenciosamente hacia producción. No obstante, SAST debe complementarse con pruebas dinámicas, ya que las vulnerabilidades de autorización dependen frecuentemente del contexto y de la lógica de negocio.

En conjunto, los laboratorios sustentan la adopción de una estrategia DevSecOps basada en defensa en profundidad, automatización y retroalimentación temprana. Una implementación profesional debería combinar configuración segura como código, análisis estático, pruebas dinámicas, casos negativos de autorización y criterios de bloqueo en CI/CD. Esta integración permite tratar la seguridad como una propiedad verificable y continua del software, en lugar de una revisión aislada al final del proyecto.

Finalmente, el trabajo aporta evidencia práctica para la revisión sistemática de la literatura al mostrar que SAST, DAST y los controles perimetrales no son técnicas excluyentes. Su valor se encuentra en la complementariedad: cada una reduce una parte distinta de la superficie de ataque y, al integrarse en un mismo flujo, mejora la capacidad de prevenir, detectar y corregir vulnerabilidades de manera temprana, reproducible y escalable.